

Le Dernier Phare

Titre de travail (EN : The Last Lighthouse) - Brief de conception et d'architecture pour Claude Code, projet Unity 6

Jeu mobile Android - roguelite court + méta-progression idle - 7 octobre 2026

Comment utiliser ce document (Claude Code). Il est la source de vérité du projet. Lis-le en entier avant de coder. Les valeurs chiffrées (PV, coûts, taux) sont des valeurs initiales : elles doivent vivre dans des ScriptableObjects et dans Remote Config, jamais en dur dans le code. Quand un point est marqué [A TRANCHER], pose la question au développeur avant d'implémenter. Le développeur est un développeur mobile expérimenté (Kotlin, Swift) mais débutant en Unity : explique brièvement les spécificités Unity (cycle de vie, ScriptableObjects, asmdef, sérialisation) quand elles comptent, sans sur-expliquer le reste.

1. Décisions figées

Sujet	Décision
Plateforme	Android d'abord. iOS possible plus tard grâce à Unity.
Moteur et langage	Unity 6 LTS, 2D avec URP, C#. IDE : JetBrains Rider (ou Visual Studio).
Titre	Le Dernier Phare (titre de travail). Nom anglais envisagé : The Last Lighthouse (vérifier les homonymes sur le Play Store avant de figer).
Genre	Roguelite court (runs de 2 à 4 min) avec méta-progression idle autour d'un archipel à restaurer.
Contrôle	Tactile direct, un doigt : le faisceau du phare suit le doigt. Un bouton d'action (Flare) sous le pouce.
Langue	Anglais au lancement (textes en anglais, via Unity Localization pour ajouter le français ensuite).
Monétisation	Pubs récompensées (cœur), interstitiels rares, achats in-app (suppression des pubs, pack de départ, pass de saison, monnaie premium, cosmétiques).
Public	Grand public, non ciblé enfants. Pas de violence graphique.
Objectif produit	Rétention quotidienne et engagement sain. Valider la rétention en soft launch avant d'investir dans le contenu et l'acquisition.

Conventions de code

- Code C# sans commentaires (les noms doivent suffire). Pas de blocs de commentaires ni de commentaires TODO dans le dépôt.
- Diffs minimaux : ne reformate pas et ne renomme pas ce qui n'est pas concerné par la tâche.
- Identifiants, noms de fichiers, clés Remote Config et événements analytics en anglais.
- La logique de jeu pure (économie, progression, stats) est en C# sans UnityEngine, testable en EditMode.
- Aucune allocation dans les boucles Update (pas de LINQ, pas de new List dans Update). Object pooling obligatoire pour ennemis, projectiles, effets.

2. Vision, univers et narration

2.1 Pitch et piliers

Le monde a été englouti en une nuit. Il ne reste que des îlots et, sur chacun, un phare éteint. Tu es le dernier gardien : en rallumant les phares, tu repousses les ténèbres marines et tu découvres pourquoi la mer est montée.

- **Lisible en deux secondes, jouable d'une main.** Un doigt pour viser, un bouton pour l'action.
- **La lumière est le langage du jeu.** Ce qui est éclairé est visible, vulnérable, vivant. Le rendu 2D éclairé fait l'essentiel de la valeur visuelle.
- **Chaque run offre des choix** (trois cartes d'amélioration) et une envie de recommencer.
- **Une progression visible chaque jour** : l'archipel se rallume îlot par îlot.
- **Monétisation respectueuse** : pubs facultatives, prix clairs, histoire jamais verrouillée derrière un paiement.

2.2 Récit (brouillon à valider)

Autrefois, un réseau de phares, la **Chain**, maintenait endormi quelque chose sous la mer, le **Deep**. Le phare mère, la **Mother Light**, s'est éteint et la mer est montée. Le joueur est le dernier gardien et progresse îlot par îlot en retrouvant des journaux de bord (fragments d'histoire trouvés pendant les runs et en restaurant les bâtiments).

- **Acte I (îles 1-2)** : comprendre que les phares forment un réseau.
- **Acte II (îles 3-4)** : découvrir le passé des gardiens (les Keepers).
- **Acte III (îles 5-6)** : la vérité. Piste : la Mother Light a été éteinte volontairement par le gardien précédent, car sa lumière nourrissait le Deep. Choix final : rallumer ou laisser dormir dans le noir (deux fins, crochet pour une saison 2).
[A TRANCHER]

2.3 Personnages

- **Wick** : compagnon, petit oiseau-lanterne. Il enseigne, commente, réagit. C'est la mascotte (icône, captures, pubs).
- **Keepers** (six, un par île) : à débloquent, chacun avec un passif et un fragment de l'histoire. Exemples : Orrin le cartographe, Sela la pêcheuse, Iven l'horloger, Nell l'apicultrice, Brann le forgeron, Mira la sonneuse de cloches. Un seul équipé par run.

2.4 Îles

#	Île	Ambiance	Nouveauté de gameplay	Boss
1	Gull's Rest	Aube calme, falaises	Tutoriel, Drifters	Tidecrab
2	Saltmarrow	Marais, brume	Lurkers (visibles seulement dans le faisceau)	Mire Warden
3	The Drowned Orchard	Vergers engloutis	Shoals (essaims rapides)	Rootmaw
4	Hollow Reef	Corail, bioluminescence	Sirens (dévient le faisceau)	The Pearl Choir
5	Brine Spires	Tempête, éclairs	Stormlings (trajectoires en zigzag)	Stormcaller
6	The Deep Gate	Abysses	Mélange de tout, boss final	The Sleeper

Vertical slice : îles 1 à 3. Lancement : îles 1 à 6 (ou 1 à 4 si le temps manque).

2.5 Direction artistique et audio

- 2D stylisé : silhouettes sombres, aplats de couleur, palette bleu nuit et ambre. Éclairage dynamique via Light 2D (Spot Light 2D avec falloff doux pour le faisceau), bloom léger.
- Lisibilité mobile : chaque ennemi se distingue par sa silhouette, pas par la couleur seule (daltonisme).
- Audio : ambiance marine en boucle, filtre passe-bas hors faisceau, sons de kill courts, carillon d'amélioration. Haptique légère sur les événements clés (option désactivable).

2.6 Glossaire (termes en jeu, anglais)

Terme	Sens
Light	Monnaie douce (runs, bâtiments)
Fragments	Monnaie de progression et d'histoire
Pearls	Monnaie premium
Lamp Oil	Énergie, une unité par run
Sparks	XP gagnée en run, remise à zéro à chaque run
Integrity	Points de vie du phare
Flare	Capacité active : pulse de lumière
Keeper / Wick	Personnage jouable à passif / compagnon mascotte

3. Gameplay

3.1 Vue d'ensemble

Vue de dessus 2D en portrait (9:16). Le phare est fixe au centre de la zone de jeu, les ennemis arrivent de tous les côtés depuis un anneau situé hors écran et avancent vers le phare. Le faisceau est un cône de lumière à 360 degrés de rotation. Tout ennemi dans le cône subit des dégâts continus. Le joueur gagne en survivant aux vagues, puis en battant le boss de l'île.

3.2 Contrôle tactile

- **Glisser n'importe où** : le faisceau pivote vers l'angle du doigt par rapport au phare, à vitesse de rotation limitée (540 degrés/s au départ) pour donner du poids.
- **Relâcher** : le faisceau continue en balayage lent (25 degrés/s) à 40 % des dégâts. Jamais zéro, pour éviter la frustration et les arrêts de jeu.
- **Bouton Flare** (zone du pouce, bas droite) : pulse circulaire, dégâts de zone et révélation des Lurkers, recharge 18 s. Option : inverser gauche/droite.
- Ignorer la visée quand le pointeur est sur l'UI (bouton Flare, pause). Utiliser l'Input System (Pointer.current) pour que le même code marche à la souris dans l'éditeur.
- Respecter la safe area (encoches) pour l'UI.

3.3 Lumière et combat

- Faisceau de base : arc 40 degrés, portée 6 unités, 20 dégâts par seconde sur chaque ennemi dans le cône. Le test d'appartenance au cône se fait par calcul d'angle et de distance (pas de Physics2D par ennemi), contre un registre d'ennemis actifs.
- **Hors lumière**, les ennemis avancent 30 % plus vite (statut Shadowed). Cela pousse à balayer au lieu de rester fixé sur un point.
- **Lurkers** : invisibles tant qu'ils ne sont pas éclairés. Un Lurker non éclairé pendant plus de 1,5 s frappe le phare.
- **Integrity** : 100 au départ. Un ennemi qui atteint le phare inflige ses dégâts de contact puis disparaît. À 0, la run est perdue.
- **Revive** : une fois par run, via pub récompensée : 50 % d'Integrity rendus et Flare automatique qui nettoie l'écran.

3.4 Ennemis (valeurs initiales)

Ennemi	PV	Vitesse	Dégâts contact	Comportement	Île
Drifter	30	1,2	5	Marche droit vers le phare	1
Lurker	45	1,6	12	Invisible hors faisceau	2
Shoal	10	2,4	2	Essaim de 6 à 10, rapide	3
Tentacle	180	0,7	20	Tank, avance lentement	3
Siren	40	0,9	0	Tant qu'elle vit, fait dériver le faisceau de 30 degrés/s	4
Stormling	35	2,0	8	Zigzag, difficile à garder dans le cône	5

Variante **Elite** (toutes îles) : x2 PV, aura visible, lâche un coffre (Fragments).

3.5 Structure d'une run

- Durée cible : 2 min 30 à 3 min 30 plus le boss. Huit vagues de 18 à 22 s, puis le boss (vague 9).
- Les vagues sont décrites dans des ScriptableObjects (WaveDefinition) : liste de groupes {ennemi, nombre, délai de départ, intervalle, motif d'apparition}.
- Les kills donnent des **Sparks**. À chaque niveau de run, le jeu se met en pause et propose **3 cartes d'amélioration** (1 choix). Un reroll par run via pub récompensée.
- XP requise pour passer du niveau n au suivant : arrondi($8 \times 1,25^{\text{puissance } n}$).
- **Boss** : PV de base 1500 multipliés par le facteur de l'île, trois phases (100, 66 et 33 %), attaques télégraphiées.
- Fin de run : victoire = récompenses complètes ; défaite = récompenses proportionnelles aux vagues terminées.

3.6 Améliorations en run (catalogue initial)

Carte	Effet par niveau	Rareté	Max	Tags
Wide Lens	+15 % d'arc	Common	4	LIGHT
Focused Lens	Arc -20 %, dégâts +35 %	Rare	3	LIGHT
Polished Mirror	+12 % de dégâts	Common	5	LIGHT
Long Wick	+20 % de portée	Common	4	LIGHT
Quick Hands	+25 % de vitesse de rotation	Common	3	LIGHT
Prism	Un second faisceau à 60 % des dégâts	Epic	1	LIGHT
Chain Light	Le faisceau saute sur une seconde cible	Epic	2	LIGHT
Ember Oil	Brûlure : 4 dégâts/s pendant 3 s	Rare	3	FIRE
Fog Shield	Ennemis proches ralentis de 25 %	Common	3	TIDE
Tide Pulse	Repoussement circulaire toutes les 8 s	Rare	3	TIDE
Gull Squadron	Mouettes alliées qui attaquent	Rare	3	ALLY
Lamp Reserve	Recharge Flare -20 %	Common	3	LIGHT
Mend	Rend 25 % d'Integrity (choix ponctuel)	Common	1	-

- Tirage : Common 60, Rare 30, Epic 10 (poids modifiables, bonus de chance côté méta). Aucune carte déjà au maximum n'est proposée.
- Tous les effets sont exprimés en StatModifier (voir section 5) : pas de logique spécifique par carte quand un modificateur de stat suffit.

3.7 Difficulté

- $PV \text{ ennemi} = PV \text{ de base} \times (1 + 0,15 \times (\text{vague} - 1)) \times \text{multiplicateur d'île}$.
- Multiplicateurs d'île : 1 ; 1,6 ; 2,5 ; 3,8 ; 5,5 ; 8.
- $\text{Nombre d'ennemis par groupe} = \text{base} \times (1 + 0,12 \times (\text{vague} - 1))$.
- **Assistance discrète** : après 3 défaites d'affilée sur une île, +10 % d'Integrity cachés jusqu'à la prochaine victoire (désactivable par Remote Config).
- Chaque île affiche une puissance recommandée (somme des niveaux méta). Pas de mur invisible : le joueur sait pourquoi il bloque et quoi améliorer.

3.8 Premières minutes (FTUE)

- Première run scriptée d'environ 90 s, guidée par Wick : glisser pour viser, utiliser Flare, premier niveau de run forcé avec une carte évidente.
- Aucune pub et aucun coût en Lamp Oil pendant les trois premières runs.
- Premier fragment d'histoire à la fin de la première victoire. Premier achat méta guidé ensuite, puis restauration d'un premier bâtiment.
- Demande de permission de notifications (Android 13 et plus) seulement après la première victoire, avec une explication claire.
- Objectif : six à dix runs lors de la première journée.

3.9 Méta-progression (hors run)

- **Pistes du phare** (niveau max initial 50) : Lens (dégâts), Lamp (gain de Light, +6 % par niveau), Hull (Integrity), Gears (vitesse de rotation), Keeper's Log (chance de raretés supérieures).
- **Restauration d'île** : bâtiments qui produisent de la Light même hors ligne (voir Économie) et débloquent l'île suivante.
- **Keepers** : passifs, équipables un par run, débloqués avec des Fragments et la progression d'île. Skins cosmétiques achetables en Pearls.
- **Performance** : 60 fps sur mobile milieu de gamme, jusqu'à 120 ennemis à l'écran.

4. Économie

Toutes les valeurs ci-dessous sont des points de départ à simuler (tableur ou script Python) avant de produire du contenu, puis à régler en soft launch via Remote Config.

4.1 Monnaies

Monnaie	Type	Sources	Utilisations
Light	Douce	Runs, Light Cache (idle), coffre quotidien	Pistes du phare, bâtiments
Fragments	Progression	Premières victoires de boss, Elites, quêtes	Keepers, chapitres d'histoire, déblocage d'îles
Pearls	Premium	Achats, goutte-à-goutte gratuit (boss premier clear 25, quêtes 10/jour, piste de saison 100)	Cosmétiques, recharge de Lamp Oil, paliers de saison
Lamp Oil	Énergie	Régénération, coffre, pubs	1 par run
Sparks	XP de run	Kills	Niveaux de run (non conservées)

4.2 Formules

```
public static class EconomyCurves
{
    public static long UpgradeCost(float baseCost, float growth, int level) =>
        (long)Math.Round(baseCost * Math.Pow(growth, level));

    public static long RunPayout(int wavesCleared, int kills, float lightBonus, float islandMult) =>
        (long)Math.Round((20 + 15 * wavesCleared + 0.4f * kills) * (1f + lightBonus) * islandMult);

    public static float EnemyHealth(float baseHealth, int wave, float islandMultiplier) =>
        baseHealth * (1f + 0.15f * (wave - 1)) * islandMultiplier;

    public static int XpToNext(int level) => (int)Math.Round(8 * Math.Pow(1.25, level));
}
```

- Coût d'une piste du phare : base 80, croissance 1,32. Niveaux 0 à 9 : 80, 106, 139, 184, 243, 321, 423, 559, 737, 973 Light (total environ 3 765 Light jusqu'au niveau 10).
- Gain de run (vague 8, 100 kills, île 1) : environ 180 Light. Multiplicateurs de gain par île : 1 ; 1,5 ; 2,2 ; 3,2 ; 4,5 ; 6.
- Bâtiment : coût base 150, croissance 1,4.

4.3 Rythme de progression visé (joueur gratuit)

Repère	Objectif
Jour 1	6 à 10 runs, 6 à 8 achats d'amélioration, première île restaurée en partie
Jour 1-2	Île 2 débloquée
Jour 3-4	Île 3
Jour 7-9	Île 4
Jour 14-18	Île 5
Jour 25-35	Île 6 et fin du premier arc narratif

4.4 Énergie (Lamp Oil)

- Maximum 5. Régénération : 1 par 25 min. Une run coûte 1. Les trois premières runs sont gratuites.
- Calculée à partir d'horodatages à la reprise de l'app, jamais avec des timers en mémoire. Prévoir une défense contre le retour en arrière de l'horloge (comparer au temps serveur Firebase quand disponible).
- Recharges bonus : pub récompensée (+1, 4 par jour maximum), coffre quotidien, 20 Pearls pour un plein. Les bonus peuvent dépasser le plafond jusqu'à 8.
- Régénération arrêtée au plafond, jamais de perte au-delà d'un bonus déjà acquis.

4.5 Production hors ligne

- Bâtiments (premiers) : **Lantern Hut** 20 Light/h, **Net Loft** (réduit la régénération d'énergie jusqu'à 20 min), **Archive** (chance de Fragments), **Dock** (qualité du coffre quotidien).
- Stockage plafonné à 8 h. À la reprise, crédit de min(temps écoulé, 8 h). Le joueur collecte la **Light Cache** ; une pub récompensée la double (3 par jour maximum).

4.6 Récompenses quotidiennes et série

- Cycle de 7 jours : J1 200 Light, J2 1 Lamp Oil, J3 20 Pearls, J4 500 Light, J5 5 Fragments, J6 2 Lamp Oil, J7 coffre (éclat cosmétique + 50 Pearls).
- Série visible sous forme de **balise** qui grandit chaque jour consécutif. Jalons à 3, 7, 14 et 30 jours avec Pearls. Un jour manqué par semaine est pardonné automatiquement (pas de remise à zéro brutale).
- Trois quêtes quotidiennes (par exemple : battre 80 ennemis, finir 3 runs, utiliser Flare 10 fois).

4.7 Publicités

Placement	Où	Récompense	Limite
revive	Écran de défaite	Reprendre à 50 % d'Integrity	1 par run
double_rewards	Écran de fin de run	Récompenses x2	Illimité
extra_oil	Hub, énergie vide	+1 Lamp Oil	4 par jour
light_cache_x2	Hub	Light Cache x2	3 par jour
reroll	Choix de cartes	Retirage des 3 cartes	1 par run

- **Interstitiels** : en fin de run seulement, jamais pendant le jeu, une run sur deux au maximum, minimum 180 s depuis la dernière pub, jamais lors des trois premières runs, jamais juste après une proposition de revive, jamais pour un joueur ayant acheté Remove Ads.
- Pas de bannière en jeu. Toute pub récompensée doit être facultative et clairement présentée comme telle.

4.8 Achats in-app

Produit (id)	Type	Prix indicatif	Contenu
remove_ads	Non consommable	4,99	Plus d'interstitiels ; coffre quotidien doublé
starter_pack	Non consommable (unique)	1,99	Proposé après la première session : 150 Pearls, 1 000 Light, 5 Lamp Oil, skin Ember
pearls_s / m / l / xl / xxl	Consommables	0,99 / 4,99 / 9,99 / 19,99 / 49,99	80 / 500 / 1 100 / 2 400 / 6 500 Pearls
season_pass	Abonnement saisonnier (achat unique par saison)	4,99	Piste premium : skins de phare, de faisceau et de Wick

- Pass de saison (4 semaines, 30 paliers) : XP via quêtes et runs. Piste gratuite : Light, Lamp Oil, 100 Pearls au total. Piste premium : cosmétiques exclusifs.
- Cosmétiques uniquement, aucune puissance de combat exclusive à l'achat (le jeu reste gagnable sans payer). Les Pearls ne s'achètent qu'avec des prix affichés en clair.

4.9 Objectifs chiffrés (à valider en soft launch)

Indicateur	Cible indicative
Rétention J1 / J7 / J30	35-40 % / 12-15 % / 4-6 %
Sessions par jour	3 ou plus ; 6 à 9 min par session
Pubs récompensées par utilisateur actif	3 à 5 par jour
Conversion payeurs	1,5 à 3 %
ARPPDAU	0,08 à 0,15 USD (ordre de grandeur de marché, non garanti)

4.10 Remote Config et analytics

Clé Remote Config	Défaut
oil_max / oil_regen_minutes	5 / 25
offline_cap_hours	8
interstitial_min_seconds / interstitial_every_n_runs	180 / 2
rewarded_extra_oil_per_day	4
upgrade_cost_base / upgrade_cost_growth	80 / 1.32
island_health_multipliers	1, 1.6, 2.5, 3.8, 5.5, 8
assist_after_losses / assist_integrity_bonus	3 / 0.10
free_runs_at_start	3

Événements analytics : tutorial_step, run_start, run_end (résultat, vague, kills, durée, île), upgrade_picked, meta_upgrade_bought, building_restored, island_unlocked, shop_open, iap_purchase, ad_rewarded_offered / started / completed, ad_interstitial_shown, energy_empty, daily_claim, streak_milestone, notification_opt_in.

Éthique du design. L'objectif d'engagement passe par la progression et les choix, pas par la manipulation : pas de faux compte à rebours, pas de probabilités cachées, pas de pénalité brutale à l'absence, pas de pub forcée au milieu d'une action. Cela réduit aussi le risque d'avis négatifs et de retrait du store.

5. Squelette d'architecture (Unity)

5.1 Paquets et réglages

Besoin	Paquet / outil
Rendu 2D	URP avec template 2D, Light 2D, Sprite Atlas
Entrées	Input System (réglage Active Input Handling : Input System Package)
Injection de dépendances	VContainer (équivalent d'esprit à Hilt)
Asynchrone	UniTask
Animations UI / tweens	DOTween ou LitMotion
Sérialisation de la sauvegarde	com.unity.nuget.newtonsoft-json (JsonUtility ne gère pas Dictionary ni HashSet)
Textes	Unity Localization, TextMeshPro
Pubs	Google Mobile Ads (AdMob) avec médiation AppLovin MAX ou LevelPlay ; consentement via Google UMP
Achats	Unity IAP (Google Play Billing)
Backend léger	Firebase : Analytics, Remote Config, Crashlytics, Cloud Messaging (External Dependency Manager pour Android)
Cloud save (plus tard)	Google Play Games Services
Tests	Unity Test Framework (EditMode et PlayMode)

- Android : IL2CPP, ARM64 (ARMv7 optionnel), format AAB, minSdk 24, targetSdk selon l'exigence Google Play au moment de la publication (à vérifier), compression de textures ASTC, orientation portrait verrouillée, Application.targetFrameRate = 60, vSync désactivé.
- Git : .gitignore Unity standard, fichiers .meta versionnés, Force Text serialization, Visible Meta Files. Git LFS pour les assets lourds (audio, textures sources).

5.2 Arborescence et assemblies

```
Assets/
  _Project/
    Art/ Audio/ Prefabs/
    Scenes/      Boot, Hub, Run
    Data/         Enemies, Upgrades, Waves, Islands, Keepers (ScriptableObjects)
    Scripts/
      Core/       Game.Core      C# pur, sans UnityEngine (noEngineReferences)
      Economy/    Game.Economy   wallet, courbes, énergie, récompenses
      Meta/       Game.Meta      progression, îles, bâtiments, keepers
      Gameplay/   Game.Gameplay  faisceau, ennemis, vagues, RunDirector
      Services/   Game.Services  interfaces + implémentations (pubs, IAP, analytics...)
      UI/         Game.UI
      Bootstrap/  Game.Bootstrap  composition root (VContainer)
    Tests/
      EditMode/   Game.Tests.EditMode
      PlayMode/   Game.Tests.PlayMode
```

- Dépendances : Economy et Meta dépendent de Core ; Gameplay dépend de Core et Economy ; UI dépend de Core, Economy, Meta, Gameplay ; Services dépend de Core ; Bootstrap dépend de tout. Jamais l'inverse.
- Scènes : **Boot** (initialise les services, charge la sauvegarde, puis charge Hub), **Hub** (méta : archipel, améliorations, boutique), **Run** (combat).
- Les services (pubs, IAP, analytics) ont toujours une implémentation Fake pour l'éditeur (#if UNITY_EDITOR) afin de tester sans SDK.

5.3 Noyau pur : portefeuille et stats

```
public enum CurrencyType { Light, Fragments, Pearls, Oil }

public sealed class Wallet
{
    private readonly Dictionary<CurrencyType, long> balances = new();
    public event Action<CurrencyType, long> Changed;

    public long Get(CurrencyType type) => balances.TryGetValue(type, out var v) ? v : 0;
```

```

public bool CanAfford(CurrencyType type, long amount) => Get(type) >= amount;

public void Add(CurrencyType type, long amount)
{
    balances[type] = Get(type) + amount;
    Changed?.Invoke(type, balances[type]);
}

public bool TrySpend(CurrencyType type, long amount)
{
    if (!CanAfford(type, amount)) return false;
    balances[type] = Get(type) - amount;
    Changed?.Invoke(type, balances[type]);
    return true;
}
}

public enum Stat { BeamDamage, BeamArc, BeamRange, TurnSpeed, MaxIntegrity, FlareCooldown, Allies }
public enum ModOp { Add, Mul }

[Serializable]
public struct StatModifier
{
    public Stat Stat;
    public ModOp Op;
    public float Value;
}

public sealed class StatBlock
{
    private readonly float[] baseValues = new float[Enum.GetValues(typeof(Stat)).Length];
    private readonly List<StatModifier> modifiers = new();

    public StatBlock(IReadOnlyDictionary<Stat, float> bases)
    {
        foreach (var pair in bases) baseValues[(int)pair.Key] = pair.Value;
    }

    public void Apply(StatModifier modifier) => modifiers.Add(modifier);

    public float Get(Stat stat)
    {
        var add = 0f;
        var mul = 1f;
        foreach (var m in modifiers)
        {
            if (m.Stat != stat) continue;
            if (m.Op == ModOp.Add) add += m.Value; else mul *= m.Value;
        }
        return (baseValues[(int)stat] + add) * mul;
    }
}

```

Pour ModOp.Mul, Value est un facteur : 1,15 signifie +15 %. Les trois niveaux de PerStack d'une carte s'appliquent donc en les ajoutant au StatBlock autant de fois que le nombre d'exemplaires pris.

5.4 Données : ScriptableObject

```

[CreateAssetMenu(menuName = "Phare/Enemy")]
public sealed class EnemyDefinition : ScriptableObject
{
    public string Id;
    public float MaxHealth;
    public float Speed;
    public float ContactDamage;
    public bool HiddenInDark;
    public int Sparks;
    public EnemyView Prefab;
}

public enum Rarity { Common, Rare, Epic }

[CreateAssetMenu(menuName = "Phare/Upgrade")]
public sealed class UpgradeDefinition : ScriptableObject
{
    public string Id;
    public Rarity Rarity;
}

```

```

    public int MaxStacks = 1;
    public string[] Tags;
    public StatModifier[] PerStack;
}

```

Prévoir aussi WaveDefinition, IslandDefinition (multiplicateurs, boss, vagues, chapitre d'histoire), KeeperDefinition, BuildingDefinition, et un GameCatalog (ScriptableObject) qui référence toutes les définitions et est injecté par le composition root.

5.5 Faisceau (contrôle tactile)

```

public static class BeamGeometry
{
    public static bool Contains(Vector2 origin, float angle, float halfArc, float range, Vector2 point)
    {
        var offset = point - origin;
        if (offset.sqrMagnitude > range * range) return false;
        var pointAngle = Mathf.Atan2(offset.y, offset.x) * Mathf.Rad2Deg;
        return Mathf.Abs(Mathf.DeltaAngle(angle, pointAngle)) <= halfArc;
    }
}

public sealed class BeamController : MonoBehaviour
{
    [SerializeField] private Transform pivot;
    [SerializeField] private float idleSweepSpeed = 25f;
    private Camera cam;
    private float angle;

    public float Angle => angle;
    public bool IsAimed { get; private set; }

    private void Awake() => cam = Camera.main;

    public void Tick(float dt, float turnSpeed)
    {
        var pointer = Pointer.current;
        IsAimed = pointer != null && pointer.press.isPressed;
        if (IsAimed)
        {
            var screen = pointer.position.ReadValue();
            var depth = -cam.transform.position.z;
            var world = cam.ScreenToWorldPoint(new Vector3(screen.x, screen.y, depth));
            var dir = world - pivot.position;
            var target = Mathf.Atan2(dir.y, dir.x) * Mathf.Rad2Deg;
            angle = Mathf.MoveTowardsAngle(angle, target, turnSpeed * dt);
        }
        else
        {
            angle += idleSweepSpeed * dt;
        }
        pivot.rotation = Quaternion.Euler(0f, 0f, angle - 90f);
    }
}

```

Le contrôleur est piloté par le RunDirector (une seule boucle Tick centrale) plutôt que par un Update par objet. Le sprite du faisceau pointe vers le haut par défaut, d'où le décalage de 90 degrés. Le cône visuel (Spot Light 2D) et le cône logique doivent partager les mêmes valeurs (arc, portée) issues du StatBlock.

5.6 Services et composition root

```

public interface IAdService
{
    bool IsRewardedReady { get; }
    UniTask<bool> ShowRewardedAsync(string placement);
    UniTask ShowInterstitialAsync(string placement);
}

public interface IPurchaseService
{
    UniTask InitializeAsync();
    UniTask<bool> PurchaseAsync(string productId);
    bool Owns(string productId);
}

public interface IAnalyticsService
{

```

```

    void Log(string eventName, params (string Key, object Value)[] parameters);
}

public interface ISaveService
{
    PlayerSave Load();
    void Save(PlayerSave data);
}

public interface IRemoteConfigService
{
    T Get<T>(string key, T fallback);
}

[Serializable]
public sealed class PlayerSave
{
    public int Version = 1;
    public Dictionary<CurrencyType, long> Balances = new();
    public Dictionary<string, int> MetaLevels = new();
    public int HighestIsland = 1;
    public long LastOilTimestamp;
    public long LastSeenTimestamp;
    public HashSet<string> OwnedProducts = new();
    public List<string> StoryFragments = new();
}

public sealed class GameLifetimeScope : LifetimeScope
{
    [SerializeField] private GameCatalog catalog;

    protected override void Configure(IContainerBuilder builder)
    {
        builder.RegisterInstance(catalog);
        builder.Register<ISaveService, JsonSaveService>(Lifetime.Singleton);
        builder.Register<IRemoteConfigService, FirebaseRemoteConfigService>(Lifetime.Singleton);
        builder.Register<IAalyticsService, FirebaseAnalyticsService>(Lifetime.Singleton);
        builder.Register<IAdService, AdMobAdService>(Lifetime.Singleton);
        builder.Register<IPurchaseService, UnityIapPurchaseService>(Lifetime.Singleton);
        builder.Register<Wallet>(Lifetime.Singleton);
        builder.Register<EnergyService>(Lifetime.Singleton);
        builder.Register<ProgressionService>(Lifetime.Singleton);
        builder.RegisterEntryPoint<BootFlow>();
    }
}

```

- **Sauvegarde** : fichier JSON dans Application.persistentDataPath, champ Version et migrations successives, écriture atomique (fichier temporaire puis remplacement), sauvegarde à chaque achat, fin de run et mise en pause de l'app (OnApplicationPause).
- **RunDirector** : machine à états Intro, Wave, UpgradeChoice, Boss, Victory, Defeat. Possède la boucle Tick, le temps de jeu (Time.timeScale à 0 pendant le choix de carte), le registre d'ennemis, le spawner avec pooling, et émet des événements C# que l'UI écoute.
- **UI** : Presenters simples (un par écran) qui observent Wallet et les services ; pas de logique métier dans les MonoBehaviours d'UI.
- **Tests EditMode** à écrire dès le début : Wallet, EconomyCurves, StatBlock, calcul de régénération d'énergie, Light Cache hors ligne, tirage de cartes (poids et plafonds).

6. Plan de réalisation

Jalon	Contenu	Critère de fin
M0 - Setup (1-2 j)	Projet Unity 6 2D URP, paquets, asmdef, Git, scènes Boot/Hub/Run, build Android qui se lance sur appareil	Écran vide qui tourne sur un vrai téléphone
M1 - Prototype (sem. 1-3)	BeamController, Drifters, spawner, Integrity, fin de run	Une run de 3 min jouable à 60 fps sur mobile milieu de gamme ; test de plaisir de jeu
M2 - Run complète	Sparks, niveaux de run, 8 cartes, 3 types d'ennemis, boss île 1, Flare, effets et sons	Run complète de bout en bout, rejouable
M3 - Méta et économie	Wallet, sauvegarde, pistes du phare, restauration îles 1-2, énergie, récompenses quotidiennes, hub	Boucle complète sur deux jours de test, tests EditMode verts

Jalon	Contenu	Critère de fin
M4 - Services	UMP, AdMob + médiation (rewarded, interstitial), IAP, Firebase (Analytics, Remote Config, Crashlytics, notifications)	Pubs et achats de test fonctionnels, événements visibles dans la console
M5 - Vertical slice	FTUE, histoire îles 1-3, localisation EN, polish, optimisation, accessibilité	Test fermé Google Play avec de vrais joueurs
M6 - Soft launch	Quelques pays, analyse de rétention et d'économie, réglages via Remote Config	Rétention J1 et J7 proches des cibles, sinon itérer avant tout lancement mondial

Publication Google Play (à vérifier au moment de publier)

- Compte développeur Play Console (frais uniques). Les comptes personnels récents peuvent devoir passer par un test fermé avec un nombre minimum de testeurs sur une durée minimale : vérifier les règles en vigueur.
- Politique de confidentialité en ligne, formulaire Data safety, questionnaire de classification, déclaration de publicité, public cible non enfant.
- Consentement UMP pour l'EEE et le Royaume-Uni avant toute demande de pub.
- Fiche en anglais : icône, feature graphic, captures, courte vidéo ; vérifier l'unicité du nom.

7. Questions ouvertes

- Nom final du jeu et vérification des homonymes. [A TRANCHER]
- Production artistique : seul, freelance, packs d'assets ? Spine ou animation par frames ? [A TRANCHER]
- Fin de l'histoire (rallumer ou laisser dormir) et nombre d'îles au lancement. [A TRANCHER]
- Budget d'acquisition d'utilisateurs, ou lancement organique seul. [A TRANCHER]
- iOS : portage prévu après validation sur Android ? [A TRANCHER]
- Langues après l'anglais (français en premier ?). [A TRANCHER]